

Production Flutter App with Provider Pattern - Complete Guide

1. Enhanced Folder Structure

```
lib/
├── core/
│   ├── constants/
│   │   ├── app_constants.dart
│   │   ├── api_constants.dart
│   │   ├── storage_constants.dart
│   │   └── route_constants.dart
│   ├── errors/
│   │   ├── app_exceptions.dart
│   │   ├── error_handler.dart
│   │   └── failure_model.dart
│   ├── network/
│   │   ├── api_client.dart
│   │   ├── api_interceptor.dart
│   │   ├── network_info.dart
│   │   └── response_model.dart
│   ├── services/
│   │   ├── storage_service.dart
│   │   ├── notification_service.dart
│   │   ├── analytics_service.dart
│   │   └── crash_reporting_service.dart
│   ├── utils/
│   │   ├── app_logger.dart
│   │   ├── date_helper.dart
│   │   ├── validation_helper.dart
│   │   ├── formatter_helper.dart
│   │   ├── debouncer.dart
│   │   └── device_info.dart
│   ├── extensions/
│   │   ├── context_extensions.dart
│   │   ├── string_extensions.dart
│   │   ├── date_extensions.dart
│   │   └── widget_extensions.dart
│   └── config/
│       ├── environment_config.dart
│       ├── app_config.dart
│       └── flavor_config.dart
├── features/
│   ├── auth/
│   │   ├── data/
│   │   │   ├── datasources/
│   │   │   │   ├── auth_local_datasource.dart
│   │   │   │   └── auth_remote_datasource.dart
│   │   │   └── models/
│   │   │       ├── user_model.dart
│   │   │       └── login_request_model.dart
```

```
| | | | └── auth_response_model.dart
| | | | └── repositories/
| | | |   └── auth_repository_impl.dart
| | | |   └── providers/
| | | |     └── auth_provider.dart
| | | └── domain/
| | |   └── entities/
| | |     └── user_entity.dart
| | |     └── auth_state_entity.dart
| | |   └── repositories/
| | |     └── auth_repository.dart
| | |   └── usecases/
| | |     └── login_usecase.dart
| | |     └── logout_usecase.dart
| | |     └── register_usecase.dart
| | |     └── refresh_token_usecase.dart
| | └── presentation/
| |   └── providers/
| |     └── auth_view_model.dart
| |     └── login_form_provider.dart
| |     └── auth_state_provider.dart
| |   └── pages/
| |     └── login_page.dart
| |     └── register_page.dart
| |     └── forgot_password_page.dart
| |     └── profile_page.dart
| |   └── widgets/
| |     └── auth_form_field.dart
| |     └── auth_button.dart
| |     └── social_login_button.dart
| |     └── password_strength_indicator.dart
| |   └── validators/
| |     └── email_validator.dart
| |     └── password_validator.dart
| |     └── form_validators.dart
| └── home/
|   └── data/
|   └── domain/
|   └── presentation/
| └── profile/
|   └── data/
|   └── domain/
|   └── presentation/
| └── settings/
|   └── data/
|   └── domain/
|   └── presentation/
```

```
└── shared/
    ├── widgets/
    │   ├── common/
    │   │   ├── custom_app_bar.dart
    │   │   ├── custom_bottom_sheet.dart
    │   │   ├── custom_dialog.dart
    │   │   ├── custom_snackbar.dart
    │   │   ├── loading_overlay.dart
    │   │   ├── error_widget.dart
    │   │   ├── empty_state_widget.dart
    │   │   ├── network_image_widget.dart
    │   │   └── shimmer_loading.dart
    │   ├── buttons/
    │   │   ├── primary_button.dart
    │   │   ├── secondary_button.dart
    │   │   ├── icon_button.dart
    │   │   └── floating_action_button.dart
    │   ├── inputs/
    │   │   ├── custom_text_field.dart
    │   │   ├── custom_dropdown.dart
    │   │   ├── custom_checkbox.dart
    │   │   ├── custom_radio.dart
    │   │   └── date_picker_field.dart
    │   ├── cards/
    │   │   ├── base_card.dart
    │   │   ├── info_card.dart
    │   │   └── list_tile_card.dart
    │   └── animations/
    │       ├── fade_animation.dart
    │       ├── slide_animation.dart
    │       └── scale_animation.dart
    ├── theme/
    │   ├── app_theme.dart
    │   ├── theme_provider.dart
    │   ├── app_colors.dart
    │   ├── app_text_styles.dart
    │   ├── app_dimensions.dart
    │   ├── app_decorations.dart
    │   └── component_themes/
    │       ├── button_theme.dart
    │       ├── input_theme.dart
    │       ├── card_theme.dart
    │       └── app_bar_theme.dart
    ├── l10n/
    │   ├── app_localizations.dart
    │   ├── app_en.arb
    │   └── app_es.arb
```

```

| | └── app_fr.arb
| | └── app_de.arb
| | └── localization_provider.dart
| └── providers/
| | └── app_provider.dart
| | └── connectivity_provider.dart
| | └── loading_provider.dart
| | └── global_state_provider.dart
| └── mixins/
| | └── validation_mixin.dart
| | └── loading_mixin.dart
| | └── error_handling_mixin.dart
└── config/
    ├── routes/
    |   ├── app_routes.dart
    |   ├── route_generator.dart
    |   ├── route_guards.dart
    |   ├── route_observer.dart
    |   └── route_transitions.dart
    ├── providers/
    |   ├── provider_setup.dart
    |   └── provider_observer.dart
    ├── di/
    |   ├── service_locator.dart
    |   └── dependency_injection.dart
    ├── environment/
    |   ├── environment.dart
    |   ├── dev_config.dart
    |   ├── staging_config.dart
    |   └── prod_config.dart
    ├── assets/
    |   ├── images/
    |   |   ├── 1.5x/
    |   |   ├── 2.0x/
    |   |   ├── 3.0x/
    |   |   └── 4.0x/
    |   ├── icons/
    |   ├── fonts/
    |   └── animations/
    └── main.dart

```

2. Advanced Route Configuration

Route Constants

dart

```
// route_constants.dart
class RouteConstants {
  static const String initial = '/';
  static const String splash = '/splash';
  static const String onboarding = '/onboarding';
  static const String login = '/login';
  static const String register = '/register';
  static const String forgotPassword = '/forgot-password';
  static const String home = '/home';
  static const String profile = '/profile';
  static const String settings = '/settings';
  static const String about = '/about';
  static const String privacyPolicy = '/privacy-policy';
  static const String termsOfService = '/terms-of-service';

  // Parameterized routes
  static const String userProfile = '/user/:userId';
  static const String productDetail = '/product/:productId';
  static const String categoryProducts = '/category/:categoryId/products';

  // Nested routes
  static const String homeTab = '/home/tab';
  static const String homeProfile = '/home/profile';
  static const String homeSettings = '/home/settings';
}
```

Route Models

dart

```
// route_model.dart
class RouteModel {
  final String path;
  final String name;
  final Widget Function(BuildContext context, Map<String, dynamic> args) builder;
  final List<RouteGuard>? guards;
  final RouteTransition? transition;
  final bool requiresAuth;
  final List<String>? requiredPermissions;

  const RouteModel({
    required this.path,
    required this.name,
    required this.builder,
    this.guards,
    this.transition,
    this.requiresAuth = false,
    this.requiredPermissions,
  });
}
```

Route Guards

dart

```
// route_guard.dart
```

```
abstract class RouteGuard {  
  Future<bool> canActivate(BuildContext context, String route);  
  String? get redirectTo;  
}  
  
class AuthGuard extends RouteGuard {  
  @override  
  Future<bool> canActivate(BuildContext context, String route) async {  
    final authProvider = Provider.of<AuthProvider>(context, listen: false);  
    return authProvider.isAuthenticated;  
  }  
  
  @override  
  String? get redirectTo => RouteConstants.login;  
}  
  
class GuestGuard extends RouteGuard {  
  @override  
  Future<bool> canActivate(BuildContext context, String route) async {  
    final authProvider = Provider.of<AuthProvider>(context, listen: false);  
    return !authProvider.isAuthenticated;  
  }  
  
  @override  
  String? get redirectTo => RouteConstants.home;  
}  
  
class PermissionGuard extends RouteGuard {  
  final String permission;  
  
  PermissionGuard(this.permission);  
  
  @override  
  Future<bool> canActivate(BuildContext context, String route) async {  
    final authProvider = Provider.of<AuthProvider>(context, listen: false);  
    return authProvider.hasPermission(permission);  
  }  
  
  @override  
  String? get redirectTo => RouteConstants.home;  
}
```



```

// route_generator.dart
class RouteGenerator {
  static final List<RouteModel> _routes = [
    RouteModel(
      path: RouteConstants.splash,
      name: 'Splash',
      builder: (context, args) => const SplashPage(),
    ),
    RouteModel(
      path: RouteConstants.login,
      name: 'Login',
      builder: (context, args) => const LoginPage(),
      guards: [GuestGuard()],
      transition: RouteTransition.slideFromRight,
    ),
    RouteModel(
      path: RouteConstants.home,
      name: 'Home',
      builder: (context, args) => const HomePage(),
      guards: [AuthGuard()],
      requiresAuth: true,
    ),
    RouteModel(
      path: RouteConstants.userProfile,
      name: 'User Profile',
      builder: (context, args) => UserProfilePage(userId: args['userId']),
      guards: [AuthGuard()],
      requiresAuth: true,
    ),
  ];

  static Route<dynamic>? generateRoute(RouteSettings settings) {
    final route = _findRoute(settings.name ?? '');
    if (route == null) {
      return _errorRoute(settings.name ?? '');
    }

    return PageRouteBuilder(
      settings: settings,
      pageBuilder: (context, animation, secondaryAnimation) {
        return FutureBuilder<bool>(
          future: _checkRouteGuards(context, route, settings.name ?? ''),
          builder: (context, snapshot) {
            if (snapshot.connectionState == ConnectionState.waiting) {
              return const LoadingPage();
            }
          }
        );
      }
    );
  }
}

```

```

        if (snapshot.hasError || !(snapshot.data ?? false)) {
            return _handleGuardFailure(context, route);
        }

        final args = _extractRouteArguments(settings);
        return route.builder(context, args);
    },
);
},
transitionsBuilder: (context, animation, secondaryAnimation, child) {
    return _buildTransition(
        route.transition ?? RouteTransition.fade,
        animation,
        secondaryAnimation,
        child,
    );
},
);
}

```

```

static RouteModel? _findRoute(String path) {
    for (final route in _routes) {
        if (_pathMatches(route.path, path)) {
            return route;
        }
    }
    return null;
}

```

```

static bool _pathMatches(String routePath, String actualPath) {
    final routeSegments = routePath.split('/');
    final actualSegments = actualPath.split('/');

    if (routeSegments.length != actualSegments.length) {
        return false;
    }

```

```

    for (int i = 0; i < routeSegments.length; i++) {
        if (routeSegments[i].startsWith(':')) {
            continue; // Parameter segment
        }
        if (routeSegments[i] != actualSegments[i]) {
            return false;
        }
    }
}

```

```

    return true;
}

static Map<String, dynamic> _extractRouteArguments(RouteSettings settings) {
    final args = <String, dynamic>{};

    // Add arguments from settings
    if (settings.arguments != null) {
        if (settings.arguments is Map<String, dynamic>) {
            args.addAll(settings.arguments as Map<String, dynamic>);
        }
    }

    // Extract path parameters
    final route = _findRoute(settings.name ?? '');
    if (route != null) {
        final routeSegments = route.path.split('/');
        final actualSegments = (settings.name ?? '').split('/');

        for (int i = 0; i < routeSegments.length; i++) {
            if (routeSegments[i].startsWith(':')) {
                final paramName = routeSegments[i].substring(1);
                args[paramName] = actualSegments[i];
            }
        }
    }

    return args;
}

static Future<bool> _checkRouteGuards(
    BuildContext context,
    RouteModel route,
    String path,
) async {
    if (route.guards == null) return true;

    for (final guard in route.guards!) {
        final canActivate = await guard.canActivate(context, path);
        if (!canActivate) {
            if (guard.redirectTo != null) {
                Navigator.of(context).pushReplacementNamed(guard.redirectTo!);
            }
            return false;
        }
    }
}

```

```
    return true;
}
```

```
static Widget _handleGuardFailure(BuildContext context, RouteModel route) {
    return const UnauthorizedPage();
}
```

```
static Widget _buildTransition(
    RouteTransition transition,
    Animation<double> animation,
    Animation<double> secondaryAnimation,
    Widget child,
) {
    switch (transition) {
        case RouteTransition.slideFromRight:
            return SlideTransition(
                position: Tween<Offset>(
                    begin: const Offset(1.0, 0.0),
                    end: Offset.zero,
                ).animate(animation),
                child: child,
            );
        case RouteTransition.slideFromLeft:
            return SlideTransition(
                position: Tween<Offset>(
                    begin: const Offset(-1.0, 0.0),
                    end: Offset.zero,
                ).animate(animation),
                child: child,
            );
        case RouteTransition.slideFromBottom:
            return SlideTransition(
                position: Tween<Offset>(
                    begin: const Offset(0.0, 1.0),
                    end: Offset.zero,
                ).animate(animation),
                child: child,
            );
        case RouteTransition.scale:
            return ScaleTransition(
                scale: Tween<double>(
                    begin: 0.0,
                    end: 1.0,
                ).animate(animation),
                child: child,
            );
        case RouteTransition.fade:
```

```

default:
  return FadeTransition(
    opacity: animation,
    child: child,
  );
}
}

static Route<dynamic> _errorRoute(String routeName) {
  return MaterialPageRoute(
    builder: (context) => ErrorPage(
      message: 'Route not found: $routeName',
    ),
  );
}
}

enum RouteTransition {
  fade,
  slideFromRight,
  slideFromLeft,
  slideFromBottom,
  scale,
}

```

Route Observer


```
// route_observer.dart
```

```
class AppRouteObserver extends NavigatorObserver {  
  static final AppRouteObserver _instance = AppRouteObserver._internal();  
  factory AppRouteObserver() => _instance;  
  AppRouteObserver._internal();  
  
  final List<Route<dynamic>> _routeStack = [];  
  
  List<Route<dynamic>> get routeStack => List.unmodifiable(_routeStack);  
  
  @override  
  void didPush(Route<dynamic> route, Route<dynamic>? previousRoute) {  
    super.didPush(route, previousRoute);  
    _routeStack.add(route);  
    _logNavigation('PUSH', route, previousRoute);  
    _trackScreenView(route);  
  }  
  
  @override  
  void didPop(Route<dynamic> route, Route<dynamic>? previousRoute) {  
    super.didPop(route, previousRoute);  
    _routeStack.remove(route);  
    _logNavigation('POP', route, previousRoute);  
    if (previousRoute != null) {  
      _trackScreenView(previousRoute);  
    }  
  }  
  
  @override  
  void didReplace({Route<dynamic>? newRoute, Route<dynamic>? oldRoute}) {  
    super.didReplace(newRoute: newRoute, oldRoute: oldRoute);  
    if (oldRoute != null) {  
      _routeStack.remove(oldRoute);  
    }  
    if (newRoute != null) {  
      _routeStack.add(newRoute);  
    }  
    _logNavigation('REPLACE', newRoute, oldRoute);  
    if (newRoute != null) {  
      _trackScreenView(newRoute);  
    }  
  }  
  
  @override  
  void didRemove(Route<dynamic> route, Route<dynamic>? previousRoute) {  
    super.didRemove(route, previousRoute);  
  }  
}
```



```

_routeStack.remove(route);
_logNavigation('REMOVE', route, previousRoute);
}

void _logNavigation(String action, Route<dynamic>? route, Route<dynamic>? previousRoute) {
  AppLogger.i(
    'Navigation $action: ${route?.settings.name} '
    '(from: ${previousRoute?.settings.name})',
  );
}

void _trackScreenView(Route<dynamic> route) {
  if (route.settings.name != null) {
    AnalyticsService.trackScreenView(route.settings.name!);
  }
}
}

```

3. Advanced Theming System

Theme Provider


```
// theme_provider.dart
```

```
class ThemeProvider extends ChangeNotifier {  
  static const String _themeKey = 'theme_mode';  
  static const String _colorSchemeKey = 'color_scheme';  
  
  ThemeMode _themeMode = ThemeMode.system;  
  AppColorScheme _colorScheme = AppColorScheme.blue;  
  bool _isSystemDark = false;  
  
  ThemeProvider() {  
    _loadThemeFromStorage();  
    _listenToSystemTheme();  
  }  
  
  // Getters  
  ThemeMode get themeMode => _themeMode;  
  AppColorScheme get colorScheme => _colorScheme;  
  bool get isDarkMode => _effectiveThemeMode == ThemeMode.dark;  
  
  ThemeMode get _effectiveThemeMode {  
    if (_themeMode == ThemeMode.system) {  
      return _isSystemDark ? ThemeMode.dark : ThemeMode.light;  
    }  
    return _themeMode;  
  }  
  
  // Theme Data  
  ThemeData get lightTheme => AppTheme.lightTheme(_colorScheme);  
  ThemeData get darkTheme => AppTheme.darkTheme(_colorScheme);  
  
  // Methods  
  Future<void> setThemeMode(ThemeMode mode) async {  
    if (_themeMode != mode) {  
      _themeMode = mode;  
      await _saveThemeToStorage();  
      notifyListeners();  
    }  
  }  
  
  Future<void> setColorScheme(AppColorScheme scheme) async {  
    if (_colorScheme != scheme) {  
      _colorScheme = scheme;  
      await _saveColorSchemeToStorage();  
      notifyListeners();  
    }  
  }  
}
```

```

void toggleTheme() {
    final newMode = _effectiveThemeMode == ThemeMode.light
        ? ThemeMode.dark
        : ThemeMode.light;
    setThemeMode(newMode);
}

Future<void> _loadThemeFromStorage() async {
    final prefs = await SharedPreferences.getInstance();
    final themeIndex = prefs.getInt(_themeKey) ?? 0;
    final colorSchemeIndex = prefs.getInt(_colorSchemeKey) ?? 0;

    _themeMode = ThemeMode.values[themeIndex];
    _colorScheme = AppColorScheme.values[colorSchemeIndex];

    notifyListeners();
}

Future<void> _saveThemeToStorage() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setInt(_themeKey, _themeMode.index);
}

Future<void> _saveColorSchemeToStorage() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setInt(_colorSchemeKey, _colorScheme.index);
}

void _listenToSystemTheme() {
    final window = WidgetsBinding.instance.platformDispatcher;
    window.onPlatformBrightnessChanged = () {
        _isSystemDark = window.platformBrightness == Brightness.dark;
        if (_themeMode == ThemeMode.system) {
            notifyListeners();
        }
    };
    _isSystemDark = window.platformBrightness == Brightness.dark;
}

```

Advanced Theme Configuration


```

// app_theme.dart
class AppTheme {
  static ThemeData lightTheme(AppColorScheme colorScheme) {
    final colors = AppColors.getColorScheme(colorScheme, false);

    return ThemeData(
      useMaterial3: true,
      brightness: Brightness.light,
      colorScheme: colors,
      scaffoldBackgroundColor: colors.background,

      // Typography
      textTheme: AppTextStyles.getTextTheme(colors, false),
      primaryTextTheme: AppTextStyles.getPrimaryTextTheme(colors, false),

      // Component Themes
      appBarTheme: AppBarThemes.lightAppBarTheme(colors),
      elevatedButtonTheme: ButtonThemes.lightElevatedButtonTheme(colors),
      outlinedButtonTheme: ButtonThemes.lightOutlinedButtonTheme(colors),
      textButtonTheme: ButtonThemes.lightTextButtonTheme(colors),
      inputDecorationTheme: InputThemes.lightInputDecorationTheme(colors),
      cardTheme: CardThemes.lightCardTheme(colors),
      dialogTheme: DialogThemes.lightDialogTheme(colors),
      bottomNavigationBarTheme: BottomNavigationBarThemes.lightTheme(colors),
      tabBarTheme: TabBarThemes.lightTabBarTheme(colors),
      chipTheme: ChipThemes.lightChipTheme(colors),
      dividerTheme: DividerThemes.lightDividerTheme(colors),

      // Extensions
      extensions: <ThemeExtension<dynamic>>[
        CustomColors.light(colorScheme),
        CustomTextStyles.light(colors),
        CustomDimensions.standard(),
      ],
    );
  }

  static ThemeData darkTheme(AppColorScheme colorScheme) {
    final colors = AppColors.getColorScheme(colorScheme, true);

    return ThemeData(
      useMaterial3: true,
      brightness: Brightness.dark,
      colorScheme: colors,
      scaffoldBackgroundColor: colors.background,

```

// Typography

```
textTheme: AppTextStyles.getTextTheme(colors, true),  
primaryTextTheme: AppTextStyles.getPrimaryTextTheme(colors, true),
```

// Component Themes

```
appBarTheme: AppBarThemes.darkAppBarTheme(colors),  
elevatedButtonTheme: ButtonThemes.darkElevatedButtonTheme(colors),  
outlinedButtonTheme: ButtonThemes.darkOutlinedButtonTheme(colors),  
textButtonTheme: ButtonThemes.darkTextButtonTheme(colors),  
inputDecorationTheme: InputThemes.darkInputDecorationTheme(colors),  
cardTheme: CardThemes.darkCardTheme(colors),  
dialogTheme: DialogThemes.darkDialogTheme(colors),  
bottomNavigationBarTheme: BottomNavigationBarThemes.darkTheme(colors),  
tabBarTheme: TabBarThemes.darkTabBarTheme(colors),  
chipTheme: ChipThemes.darkChipTheme(colors),  
dividerTheme: DividerThemes.darkDividerTheme(colors),
```

// Extensions

```
extensions: <ThemeExtension<dynamic>>[  
  CustomColors.dark(colorScheme),  
  CustomTextStyles.dark(colors),  
  CustomDimensions.standard(),  
,  
>;  
}  
}
```

Color System


```
// app_colors.dart
```

```
enum AppColorScheme {  
  blue,  
  green,  
  purple,  
  orange,  
  red,  
  teal,  
}
```

```
class AppColors {  
  static ColorScheme getColorScheme(AppColorScheme scheme, bool isDark) {  
    switch (scheme) {  
      case AppColorScheme.blue:  
        return isDark ? _blueDarkScheme : _blueLightScheme;  
      case AppColorScheme.green:  
        return isDark ? _greenDarkScheme : _greenLightScheme;  
      case AppColorScheme.purple:  
        return isDark ? _purpleDarkScheme : _purpleLightScheme;  
      case AppColorScheme.orange:  
        return isDark ? _orangeDarkScheme : _orangeLightScheme;  
      case AppColorScheme.red:  
        return isDark ? _redDarkScheme : _redLightScheme;  
      case AppColorScheme.teal:  
        return isDark ? _tealDarkScheme : _tealLightScheme;  
    }  
  }  
}
```

```
// Blue Color Scheme
```

```
static const _blueLightScheme = ColorScheme(  
  brightness: Brightness.light,  
  primary: Color(0xFF1976D2),  
  onPrimary: Color(0xFFFFFFFF),  
  primaryContainer: Color(0xFFBBDEFB),  
  onPrimaryContainer: Color(0xFF0D47A1),  
  secondary: Color(0xFF03DAC6),  
  onSecondary: Color(0xFF000000),  
  secondaryContainer: Color(0xFFE0F2F1),  
  onSecondaryContainer: Color(0xFF00695C),  
  tertiary: Color(0xFF9C27B0),  
  onTertiary: Color(0xFFFFFFFF),  
  tertiaryContainer: Color(0xFFE1BEE7),  
  onTertiaryContainer: Color(0xFF4A148C),  
  error: Color(0xFFBA1A1A),  
  onError: Color(0xFFFFFFFF),  
  errorContainer: Color(0xFFFFDAD6),
```

```
onErrorContainer: Color(0xFF410E0B),
background: Color(0xFFFFAFAFA),
onBackground: Color(0xFF1C1B1F),
surface: Color(0xFFFFFFFF),
onSurface: Color(0xFF1C1B1F),
surfaceVariant: Color(0xFFE7E0EC),
onSurfaceVariant: Color(0xFF49454F),
outline: Color(0xFF79747E),
outlineVariant: Color(0xFFCAC4D0),
shadow: Color(0xFF000000),
scrim: Color(0xFF000000),
inverseSurface: Color(0xFF313033),
onInverseSurface: Color(0xFFFF4EFF4),
inversePrimary: Color(0xFF90CAF9),
);
```

```
static const _blueDarkScheme = ColorScheme(
  brightness: Brightness.dark,
  primary: Color(0xFF90CAF9),
  onPrimary: Color(0xFF0D47A1),
  primaryContainer: Color(0xFF1565C0),
  onPrimaryContainer: Color(0xFFBBDEFB),
  secondary: Color(0xFF4DD0E1),
  onSecondary: Color(0xFF00695C),
  secondaryContainer: Color(0xFF00838F),
  onSecondaryContainer: Color(0xFFE0F2F1),
  tertiary: Color(0xFFCE93D8),
  onTertiary: Color(0xFF4A148C),
  tertiaryContainer: Color(0xFF7B1FA2),
  onTertiaryContainer: Color(0xFFE1BEE7),
  error: Color(0xFFFFB4AB),
  onError: Color(0xFF690005),
  errorContainer: Color(0xFF93000A),
  onErrorContainer: Color(0xFFFFDAD6),
  background: Color(0xFF121212),
  onBackground: Color(0xFFE6E1E5),
  surface: Color(0xFF1E1E1E),
  onSurface: Color(0xFFE6E1E5),
  surfaceVariant: Color(0xFF49454F),
  onSurfaceVariant: Color(0xFFCAC4D0),
  outline: Color(0xFF938F99),
  outlineVariant: Color(0xFF49454F),
  shadow: Color(0xFF000000),
  scrim: Color(0xFF000000),
  inverseSurface: Color(0xFFE6E1E5),
  onInverseSurface: Color(0xFF313033),
  inversePrimary: Color(0xFF1976D2),
```

```
);
```

```
// Additional color schemes would be defined here...
```

```
// For brevity, showing structure for other schemes
```

```
// Semantic Colors
```

```
static const Color success = Color(0xFF4CAF50);
```

```
static const Color successDark = Color(0xFF388E3C);
```

```
static const Color warning = Color(0xFFFF9800);
```

```
static const Color warningDark = Color(0xFFFF57C0);
```

```
static const Color info = Color(0xFF2196F3);
```

```
static const Color infoDark = Color(0xFF1976D2);
```

```
// Neutral Colors
```

```
static const Color grey50 = Color(0xFFFAFAFA);
```

```
static const Color grey100 = Color(0xFFFF5F5F5);
```

```
static const Color grey200 = Color(0xFFEEEEEE);
```

```
static const Color grey300 = Color(0xFFE0E0E0);
```

```
static const Color grey400 = Color(0xFFBDBDBD);
```

```
static const Color grey500 = Color(0xFF9E9E9E);
```

```
static const Color grey600 = Color(0xFF757575);
```

```
static const Color grey700 = Color(0xFF616161);
```

```
static const Color grey800 = Color(0xFF424242);
```

```
static const Color grey900 = Color(0xFF212121);
```

```
}
```

```
// Custom Colors Theme Extension
```

```
@immutable
```

```
class CustomColors extends ThemeExtension<CustomColors> {
```

```
  final Color success;
```

```
  final Color warning;
```

```
  final Color info;
```

```
  final Color successContainer;
```

```
  final Color warningContainer;
```

```
  final Color infoContainer;
```

```
  final Color onSuccess;
```

```
  final Color onWarning;
```

```
  final Color onInfo;
```

```
const CustomColors({
```

```
  required this.success,
```

```
  required this.warning,
```

```
  required this.info,
```

```
  required this.successContainer,
```

```
  required this.warningContainer,
```

```
  required this.infoContainer,
```

```
  required this.onSuccess,
```

```
required this.onWarning,  
required this.onInfo,  
});
```

```
CustomColors.light(AppColorScheme scheme) : this(  
  success: AppColors.success,  
  warning: AppColors.warning,  
  info: AppColors.info,  
  successContainer: AppColors.success.withOpacity(0.12),  
  warningContainer: AppColors.warning.withOpacity(0.12),  
  infoContainer: AppColors.info.withOpacity(0.12),  
  onSuccess: Colors.white,  
  onWarning: Colors.white,  
  onInfo: Colors.white,  
);
```

```
CustomColors.dark(AppColorScheme scheme) : this(  
  success: AppColors.successDark,  
  warning: AppColors.warningDark,  
  info: AppColors.infoDark,  
  successContainer: AppColors.successDark.withOpacity(0.12),  
  warningContainer: AppColors.warningDark.withOpacity(0.12),  
  infoContainer: AppColors.infoDark.withOpacity(0.12),  
  onSuccess: Colors.white,  
  onWarning: Colors.white,  
  onInfo: Colors.white,  
);
```

@override

```
CustomColors.copyWith({  
  Color? success,  
  Color? warning,  
  Color? info,  
  Color? successContainer,  
  Color? warningContainer,  
  Color? infoContainer,  
  Color? onSuccess,  
  Color? onWarning,  
  Color? onInfo,  
}) {  
  return CustomColors(  
    success: success ?? this.success,  
    warning: warning ?? this.warning,  
    info: info ?? this.info,  
    successContainer: successContainer ?? this.successContainer,  
    warningContainer: warningContainer ?? this.warningContainer,  
    infoContainer: infoContainer ?? this.infoContainer,
```

```
onSuccess: onSuccess ?? this.onSuccess,  
onWarning: onWarning ?? this.onWarning,  
onInfo: onInfo ?? this.onInfo,  
);  
}
```

@override

```
CustomColors lerp(ThemeExtension<CustomColors>? other, double t) {  
  if (other is! CustomColors) return this;  
  return CustomColors(  
    success: Color.lerp(success, other.success, t)!,  
    warning: Color.lerp(warning, other.warning, t)!,  
    info: Color.lerp(info, other.info, t)!,  
    successContainer: Color.lerp(successContainer, other.successContainer, t)!,  
    warningContainer: Color.lerp(warningContainer, other.warningContainer, t)!,  
    infoContainer: Color.lerp(infoContainer, other.infoContainer, t)!,  
    onSuccess: Color.lerp(onSuccess, other.onSuccess, t)!,  
    onWarning: Color.lerp(onWarning, other.onWarning, t)!,  
    onInfo: Color.lerp(onInfo, other.onInfo, t)!,  
  );  
}
```

Typography System


```
// app_text_styles.dart
```

```
class AppTextStyles {  
  static TextTheme getTextTheme(ColorScheme colorScheme, bool isDark) {  
    return TextTheme(  
      displayLarge: GoogleFonts.inter(  
        fontSize: 57,  
        fontWeight: FontWeight.w400,  
        letterSpacing: -0.25,  
        color: colorScheme.onSurface,  
      ),  
      displayMedium: GoogleFonts.inter(  
        fontSize: 45,  
        fontWeight: FontWeight.w400,  
        letterSpacing: 0,  
        color: colorScheme.onSurface,  
      ),  
      displaySmall: GoogleFonts.inter(  
        fontSize: 36,  
        fontWeight: FontWeight.w400,  
        letterSpacing: 0,  
        color: colorScheme.onSurface,  
      ),  
      headlineLarge: GoogleFonts.inter(  
        fontSize: 32,  
        fontWeight: FontWeight.w400,  
        letterSpacing: 0,  
        color: colorScheme.onSurface,  
      ),  
      headlineMedium: GoogleFonts.inter(  
        fontSize: 28,  
        fontWeight: FontWeight.w400,  
        letterSpacing: 0,  
        color: colorScheme.onSurface,  
      ),  
      headlineSmall: GoogleFonts.inter(  
        fontSize: 24,  
        fontWeight: FontWeight.w400,  
        letterSpacing: 0,  
        color: colorScheme.onSurface,  
      ),  
      titleLarge: GoogleFonts.inter(  
        fontSize: 22,  
        fontWeight: FontWeight.w500,  
        letterSpacing: 0,  
        color: colorScheme.onSurface,  
      ),  
    );  
  }  
}
```

```
titleMedium: GoogleFonts.inter(  
  fontSize: 16,  
  fontWeight: FontWeight.w500,  
  letterSpacing: 0.15,  
  color: colorScheme.onSurface,  
)  
titleSmall: GoogleFonts.inter(  
  fontSize: 14,  
  fontWeight: FontWeight.w500,  
  letterSpacing: 0.1,  
  color: colorScheme.onSurface,  
)  
bodyLarge: GoogleFonts.inter(  
  fontSize: 16,  
  fontWeight: FontWeight.w400,  
  letterSpacing: 0.5,  
  color: colorScheme.onSurface,  
)  
bodyMedium: GoogleFonts.inter(  
  fontSize: 14,  
  fontWeight: FontWeight.w400,  
  letterSpacing: 0.25,  
  color: colorScheme.onSurface,  
)  
bodySmall: GoogleFonts.inter(  
  fontSize: 12,  
  fontWeight: FontWeight.w400,  
  letterSpacing: 0.4,  
  color: colorScheme.onSurface,  
)  
labelLarge: GoogleFonts.inter(  
  fontSize: 14,  
  fontWeight: FontWeight.w500,  
  letterSpacing: 0.1,  
  color: colorScheme.onSurface,  
)  
labelMedium: GoogleFonts.inter(  
  fontSize: 12,  
  fontWeight: FontWeight.w500,  
  letterSpacing: 0.5,  
  color: colorScheme.onSurface,  
)  
labelSmall: GoogleFonts.inter(  
  fontSize: 11,  
  fontWeight: FontWeight.w500,  
  letterSpacing: 0.5,  
  color: colorScheme.onSurface,
```



```
),  
);  
}
```

```
static TextTheme getPrimaryTextTheme(ColorScheme colorScheme, bool isDark) {  
  return getTextTheme(colorScheme, isDark).apply(  
    bodyColor: colorScheme.onPrimary,  
    displayColor: colorScheme.onPrimary,  
  );  
}  
}
```

// Custom Text Styles Extension

@immutable

```
class CustomTextStyles extends ThemeExtension<CustomTextStyles> {  
  final TextStyle caption;  
  final TextStyle overline;  
  final TextStyle button;  
  final TextStyle subtitle;  
  final TextStyle code;
```

```
  const CustomTextStyles({  
    required this.caption,  
    required this.overline,  
    required this.button,  
    required this.subtitle,  
    required this.code,  
  });
```

```
  CustomTextStyles.light(ColorScheme colorScheme) : this(  
    caption: GoogleFonts.inter(  
      fontSize: 12,  
      fontWeight: FontWeight.w400,  
      letterSpacing: 0.4,  
      color: colorScheme.onSurface.withOpacity(0.6),  
    ),  
    overline: GoogleFonts.inter(  
      fontSize: 10,  
      fontWeight: FontWeight.w500,  
      letterSpacing: 1.5,  
      color: colorScheme.onSurface.withOpacity(0.6),  
    ),  
    button: GoogleFonts.inter(  
      fontSize: 14,  
      fontWeight: FontWeight.w500,  
      letterSpacing: 1.25,  
      color: colorScheme.primary,
```

```
),
subtitle: GoogleFonts.inter(
  fontSize: 16,
  fontWeight: FontWeight.w400,
  letterSpacing: 0.15,
  color: colorScheme.onSurface.withOpacity(0.6),
),
code: GoogleFonts.jetBrainsMono(
  fontSize: 14,
  fontWeight: FontWeight.w400,
  letterSpacing: 0,
  color: colorScheme.onSurface,
),
);
```

```
CustomTextStyles.dark(ColorScheme colorScheme) : this(
  caption: GoogleFonts.inter(
    fontSize: 12,
    fontWeight: FontWeight.w400,
    letterSpacing: 0.4,
    color: colorScheme.onSurface.withOpacity(0.6),
  ),
  overline: GoogleFonts.inter(
    fontSize: 10,
    fontWeight: FontWeight.w500,
    letterSpacing: 1.5,
    color: colorScheme.onSurface.withOpacity(0.6),
  ),
  button: GoogleFonts.inter(
    fontSize: 14,
    fontWeight: FontWeight.w500,
    letterSpacing: 1.25,
    color: colorScheme.primary,
  ),
  subtitle: GoogleFonts.inter(
    fontSize: 16,
    fontWeight: FontWeight.w400,
    letterSpacing: 0.15,
    color: colorScheme.onSurface.withOpacity(0.6),
  ),
  code: GoogleFonts.jetBrainsMono(
    fontSize: 14,
    fontWeight: FontWeight.w400,
    letterSpacing: 0,
    color: colorScheme.onSurface,
  ),
);
```

```

@override
CustomTextStyles copyWith({
  TextStyle? caption,
  TextStyle? overline,
  TextStyle? button,
  TextStyle? subtitle,
  TextStyle? code,
}) {
  return CustomTextStyles(
    caption: caption ?? this.caption,
    overline: overline ?? this.overline,
    button: button ?? this.button,
    subtitle: subtitle ?? this.subtitle,
    code: code ?? this.code,
  );
}

```

```

@override
CustomTextStyles lerp(ThemeExtension<CustomTextStyles>? other, double t) {
  if (other is! CustomTextStyles) return this;
  return CustomTextStyles(
    caption: TextStyle.lerp(caption, other.caption, t)!,
    overline: TextStyle.lerp(overline, other.overline, t)!,
    button: TextStyle.lerp(button, other.button, t)!,
    subtitle: TextStyle.lerp(subtitle, other.subtitle, t)!,
    code: TextStyle.lerp(code, other.code, t)!,
  );
}
}

```

Dimensions System


```
// app_dimensions.dart
```

```
@immutable
```

```
class CustomDimensions extends ThemeExtension<CustomDimensions> {
```

```
  final double paddingSmall;
```

```
  final double paddingMedium;
```

```
  final double paddingLarge;
```

```
  final double paddingXLarge;
```

```
  final double marginSmall;
```

```
  final double marginMedium;
```

```
  final double marginLarge;
```

```
  final double marginXLarge;
```

```
  final double radiusSmall;
```

```
  final double radiusMedium;
```

```
  final double radiusLarge;
```

```
  final double radiusXLarge;
```

```
  final double elevationSmall;
```

```
  final double elevationMedium;
```

```
  final double elevationLarge;
```

```
  const CustomDimensions({
```

```
    required this.paddingSmall,
```

```
    required this.paddingMedium,
```

```
    required this.paddingLarge,
```

```
    required this.paddingXLarge,
```

```
    required this.marginSmall,
```

```
    required this.marginMedium,
```

```
    required this.marginLarge,
```

```
    required this.marginXLarge,
```

```
    required this.radiusSmall,
```

```
    required this.radiusMedium,
```

```
    required this.radiusLarge,
```

```
    required this.radiusXLarge,
```

```
    required this.elevationSmall,
```

```
    required this.elevationMedium,
```

```
    required this.elevationLarge,
```

```
  });
```

```
CustomDimensions.standard() : this(
```

```
  paddingSmall: 8.0,
```

```
  paddingMedium: 16.0,
```

```
  paddingLarge: 24.0,
```

```
  paddingXLarge: 32.0,
```

```
  marginSmall: 8.0,
```

```
  marginMedium: 16.0,
```

```
  marginLarge: 24.0,
```

```
  marginXLarge: 32.0,
```

```
radiusSmall: 4.0,  
radiusMedium: 8.0,  
radiusLarge: 12.0,  
radiusXLarge: 16.0,  
elevationSmall: 2.0,  
elevationMedium: 4.0,  
elevationLarge: 8.0,  
);
```

@override

```
CustomDimensions copyWith({  
  double? paddingSmall,  
  double? paddingMedium,  
  double? paddingLarge,  
  double? paddingXLarge,  
  double? marginSmall,  
  double? marginMedium,  
  double? marginLarge,  
  double? marginXLarge,  
  double? radiusSmall,  
  double? radiusMedium,  
  double? radiusLarge,  
  double? radiusXLarge,  
  double? elevationSmall,  
  double? elevationMedium,  
  double? elevationLarge,  
}) {  
  return CustomDimensions(  
    paddingSmall: paddingSmall ?? this.paddingSmall,  
    paddingMedium: paddingMedium ?? this.paddingMedium,  
    paddingLarge: paddingLarge ?? this.paddingLarge,  
    paddingXLarge: paddingXLarge ?? this.paddingXLarge,  
    marginSmall: marginSmall ?? this.marginSmall,  
    marginMedium: marginMedium ?? this.marginMedium,  
    marginLarge: marginLarge ?? this.marginLarge,  
    marginXLarge: marginXLarge ?? this.marginXLarge,  
    radiusSmall: radiusSmall ?? this.radiusSmall,  
    radiusMedium: radiusMedium ?? this.radiusMedium,  
    radiusLarge: radiusLarge ?? this.radiusLarge,  
    radiusXLarge: radiusXLarge ?? this.radiusXLarge,  
    elevationSmall: elevationSmall ?? this.elevationSmall,  
    elevationMedium: elevationMedium ?? this.elevationMedium,  
    elevationLarge: elevationLarge ?? this.elevationLarge,  
  );  
}
```

@override

```

CustomDimensions lerp(ThemeExtension<CustomDimensions>? other, double t) {
    if (other is! CustomDimensions) return this;
    return CustomDimensions(
        paddingSmall: lerpDouble(paddingSmall, other.paddingSmall, t)!,
        paddingMedium: lerpDouble(paddingMedium, other.paddingMedium, t)!,
        paddingLarge: lerpDouble(paddingLarge, other.paddingLarge, t)!,
        paddingXLarge: lerpDouble(paddingXLarge, other.paddingXLarge, t)!,
        marginSmall: lerpDouble(marginSmall, other.marginSmall, t)!,
        marginMedium: lerpDouble(marginMedium, other.marginMedium, t)!,
        marginLarge: lerpDouble(marginLarge, other.marginLarge, t)!,
        marginXLarge: lerpDouble(marginXLarge, other.marginXLarge, t)!,
        radiusSmall: lerpDouble(radiusSmall, other.radiusSmall, t)!,
        radiusMedium: lerpDouble(radiusMedium, other.radiusMedium, t)!,
        radiusLarge: lerpDouble(radiusLarge, other.radiusLarge, t)!,
        radiusXLarge: lerpDouble(radiusXLarge, other.radiusXLarge, t)!,
        elevationSmall: lerpDouble(elevationSmall, other.elevationSmall, t)!,
        elevationMedium: lerpDouble(elevationMedium, other.elevationMedium, t)!,
        elevationLarge: lerpDouble(elevationLarge, other.elevationLarge, t)!,
    );
}

```

4. Advanced Localization System

Localization Provider


```

// localization_provider.dart

class LocalizationProvider extends ChangeNotifier {
  static const String _localeKey = 'locale';

  Locale _locale = const Locale('en');
  bool _isLoading = false;

  Locale get locale => _locale;
  bool get isLoading => _isLoading;

  List<Locale> get supportedLocales => const [
    Locale('en', 'US'),
    Locale('es', 'ES'),
    Locale('fr', 'FR'),
    Locale('de', 'DE'),
    Locale('it', 'IT'),
    Locale('pt', 'PT'),
    Locale('ru', 'RU'),
    Locale('zh', 'CN'),
    Locale('ja', 'JP'),
    Locale('ko', 'KR'),
    Locale('ar', 'SA'),
    Locale('hi', 'IN'),
  ];

  LocalizationProvider() {
    _loadLocaleFromStorage();
  }

  Future<void> setLocale(Locale locale) async {
    if (_locale != locale && supportedLocales.contains(locale)) {
      _isLoading = true;
      notifyListeners();

      _locale = locale;
      await _saveLocaleToStorage();

      // Preload localization data
      await _preloadLocalizationData(locale);

      _isLoading = false;
      notifyListeners();
    }
  }

  Future<void> setLocaleByCode(String languageCode, [String? countryCode]) async {

```

```

final locale = Locale(languageCode, countryCode);
await setLocale(locale);
}

```

```

Future<void> _loadLocaleFromStorage() async {
  final prefs = await SharedPreferences.getInstance();
  final languageCode = prefs.getString('${_localeKey}_language') ?? 'en';
  final countryCode = prefs.getString('${_localeKey}_country');

  final locale = Locale(languageCode, countryCode);
  if (supportedLocales.contains(locale)) {
    _locale = locale;
  } else {
    // Fallback to system locale if supported
    final systemLocale = PlatformDispatcher.instance.locale;
    if (supportedLocales.contains(systemLocale)) {
      _locale = systemLocale;
    }
  }

  notifyListeners();
}

```

```

Future<void> _saveLocaleToStorage() async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('${_localeKey}_language', _locale.languageCode);
  if (_locale.countryCode != null) {
    await prefs.setString('${_localeKey}_country', _locale.countryCode!);
  }
}

```

```

Future<void> _preloadLocalizationData(Locale locale) async {
  // Implement preloading logic if needed
  // This could involve caching translations, etc.
  await Future.delayed(const Duration(milliseconds: 300));
}

```

```

String getLocalizedCountryName(String countryCode) {
  // Implement country name localization
  final countryNames = {
    'en': {
      'US': 'United States',
      'ES': 'Spain',
      'FR': 'France',
      'DE': 'Germany',
      // Add more countries
    },
  },

```

```

'es': {
    'US': 'Estados Unidos',
    'ES': 'España',
    'FR': 'Francia',
    'DE': 'Alemania',
    // Add more countries
},
// Add more languages
};

return countryNames[_locale.languageCode]?[countryCode] ?? countryCode;
}

```

```

String getLocalizedLanguageName(String languageCode) {

```

```

    final languageNames = {

```

```

        'en': {
            'en': 'English',
            'es': 'Spanish',
            'fr': 'French',
            'de': 'German',
            'it': 'Italian',
            'pt': 'Portuguese',
            'ru': 'Russian',
            'zh': 'Chinese',
            'ja': 'Japanese',
            'ko': 'Korean',
            'ar': 'Arabic',
            'hi': 'Hindi',

```

```

        },

```

```

        'es': {
            'en': 'Inglés',
            'es': 'Español',
            'fr': 'Francés',
            'de': 'Alemán',
            'it': 'Italiano',
            'pt': 'Portugués',
            'ru': 'Ruso',
            'zh': 'Chino',
            'ja': 'Japonés',
            'ko': 'Coreano',
            'ar': 'Árabe',
            'hi': 'Hindi',

```

```

        },

```

```

        // Add more languages

```

```

    };

```

```

    return languageNames[_locale.languageCode]?[languageCode] ?? languageCode;

```

}

}

Enhanced ARB Files


```
// app_en.arb
{
  "@@locale": "en",
  "@@last_modified": "2024-01-15T10:00:00",

  "appTitle": "My Production App",
  "@appTitle": {
    "description": "The title of the application"
  },

  "welcome": "Welcome",
  "@welcome": {
    "description": "Welcome message"
  },

  "welcomeUser": "Welcome, {name}!",
  "@welcomeUser": {
    "description": "Welcome message with user name",
    "placeholders": {
      "name": {
        "type": "String",
        "example": "John"
      }
    }
  },

  "itemCount": "{count, plural, =0{No items} =1{One item} other{{count} items}}",
  "@itemCount": {
    "description": "Number of items with pluralization",
    "placeholders": {
      "count": {
        "type": "int",
        "example": "5"
      }
    }
  },

  "dateFormat": "{date}",
  "@dateFormat": {
    "description": "Date format",
    "placeholders": {
      "date": {
        "type": "DateTime",
        "format": "yMMMd"
      }
    }
  }
}
```

```

},

"timeFormat": "{time}",
"@timeFormat": {
  "description": "Time format",
  "placeholders": {
    "time": {
      "type": "DateTime",
      "format": "jm"
    }
  }
},

"currency": "{amount}",
"@currency": {
  "description": "Currency format",
  "placeholders": {
    "amount": {
      "type": "double",
      "format": "currency",
      "optionalParameters": {
        "symbol": "$"
      }
    }
  }
},

"percentage": "{value}",
"@percentage": {
  "description": "Percentage format",
  "placeholders": {
    "value": {
      "type": "double",
      "format": "percentPattern"
    }
  }
},

"genderMessage": "{gender, select, male{He is a user} female{She is a user} other{They are a user}}",
"@genderMessage": {
  "description": "Message with gender selection",
  "placeholders": {
    "gender": {
      "type": "String"
    }
  }
},

```

"login": "Login",
"logout": "Logout",
"register": "Register",
"forgotPassword": "Forgot Password?",
"resetPassword": "Reset Password",
"changePassword": "Change Password",
"confirmPassword": "Confirm Password",
"email": "Email",
"password": "Password",
"newPassword": "New Password",
"currentPassword": "Current Password",
"firstName": "First Name",
"lastName": "Last Name",
"phoneNumber": "Phone Number",
"address": "Address",
"city": "City",
"state": "State",
"zipCode": "ZIP Code",
"country": "Country",
"dateOfBirth": "Date of Birth",
"gender": "Gender",
"male": "Male",
"female": "Female",
"other": "Other",
"preferNotToSay": "Prefer not to say",

"save": "Save",
"cancel": "Cancel",
"delete": "Delete",
"edit": "Edit",
"add": "Add",
"remove": "Remove",
"update": "Update",
"refresh": "Refresh",
"retry": "Retry",
"back": "Back",
"next": "Next",
"previous": "Previous",
"finish": "Finish",
"skip": "Skip",
"done": "Done",
"close": "Close",
"ok": "OK",
"yes": "Yes",
"no": "No",
"confirm": "Confirm",


```
"search": "Search",
"filter": "Filter",
"sort": "Sort",
"clear": "Clear",
"apply": "Apply",
"reset": "Reset",

"loading": "Loading...",
"pleaseWait": "Please wait...",
"processing": "Processing...",
"saving": "Saving...",
"updating": "Updating...",
"deleting": "Deleting...",
"uploading": "Uploading...",
"downloading": "Downloading...",

"success": "Success",
"error": "Error",
"warning": "Warning",
"info": "Information",
"notice": "Notice",

"networkError": "Network error. Please check your connection.",
"serverError": "Server error. Please try again later.",
"unknownError": "An unknown error occurred.",
"validationError": "Please check your input and try again.",
"authenticationError": "Authentication failed. Please login again.",
"authorizationError": "You don't have permission to perform this action.",
"notFoundError": "The requested resource was not found.",
"timeoutError": "Request timeout. Please try again.",

"invalidEmail": "Please enter a valid email address.",
"invalidPassword": "Password must be at least 8 characters long.",
"passwordMismatch": "Passwords do not match.",
"requiredField": "This field is required.",
"invalidPhoneNumber": "Please enter a valid phone number.",
"invalidUrl": "Please enter a valid URL.",
"invalidDate": "Please enter a valid date.",
"minLength": "Minimum length is {min} characters.",
"@minLength": {
  "placeholders": {
    "min": {
      "type": "int"
    }
  }
},
"maxLength": "Maximum length is {max} characters.",
```

```
"@maxLength": {  
  "placeholders": {  
    "max": {  
      "type": "int"  
    }  
  }  
},  

```

```
"home": "Home",  
"profile": "Profile",  
"settings": "Settings",  
"notifications": "Notifications",  
"favorites": "Favorites",  
"history": "History",  
"help": "Help",  
"support": "Support",  
"about": "About",  
"feedback": "Feedback",  
"contactUs": "Contact Us",  
"privacyPolicy": "Privacy Policy",  
"termsOfService": "Terms of Service",  
"version": "Version",  
"buildNumber": "Build Number",  

```

```
"theme": "Theme",  
"language": "Language",  
"darkMode": "Dark Mode",  
"lightMode": "Light Mode",  
"systemMode": "System Mode",  
"colorScheme": "Color Scheme",  
"fontSize": "Font Size",  
"fontFamily": "Font Family",  

```

```
"emptyStateTitle": "Nothing here yet",  
"emptyStateMessage": "When you have items, they'll appear here.",  
"emptyStateAction": "Add your first item",  

```